Embedding Live Machine Learning in Csound: Emotion-Driven Harmony via an ONNX-Runtime Opcode

AuthorA[1] and AuthorB[2]

[1] InstituteA
[2] InstituteB
your.email@yourdomain.com

Abstract. Harmonic generation in Csound typically requires explicit chord specification in the score or coordination with an external process. We present the *Affective Harmony Machine* (AHM), a system that embeds an ONNX-format machine-learning model as a native Csound opcode, demonstrated through affect-driven chord progression generation. The system is backed by an annotated dataset of 108 jazz and pop chord progressions labeled with six emotion classes and seven scale/mode categories. A Random Forest classifier trained on this dataset is exported to the ONNX interchange format. The authors created `emoChord`, a Csound opcode that accepts an emotion string, performs live ML inference at i-rate via the ONNX Runtime C API, and schedules harmonically appropriate chord events into a synthesis instrument. No Python interpreter or external process is required at runtime; the current release targets macOS (arm64 natively, x86_64 via Rosetta). The implementation demonstrates a general pattern for embedding any ONNX-compatible ML model in Csound as a native plugin with no Python runtime requirement.

Keywords: Csound opcode, ONNX Runtime, machine learning, harmony generation, emotion, chord progression, affective computing

1 Introduction

Csound [7] has a long tradition of algorithmic composition, from Lorrain's stochastic cannons [2] to contemporary generative harmony and algorithmic interactive systems. Yet in most of these systems, harmony remains the composer's explicit responsibility: chords are either written into the score by hand or drawn from probability tables constructed in advance. Shi and Boulanger's CStore [1] demonstrated AI-driven orchestra code from natural language, though as an external process, sharpening the question: how far can machine intelligence be embedded *within* Csound itself?

We answer concretely: any ONNX-compatible classifier following the same input/output tensor contract can be embedded as a native Csound i-rate opcode via the ONNX Runtime C API with no external-process dependency. As a demonstration, the *Affective Harmony Machine* (AHM) targets affect-driven harmony, where emotional states map to musical parameters such as mode and tension [**?**]: a composer writes `i1 0 4 "joyful"` to receive chord events scheduled into any synthesis instrument.

2 Dataset

The system is trained on two curated harmony corpora. The jazz dataset covers functional harmony, including secondary dominants, tritone substitutions, and deceptive resolutions, across 25 unique progressions, each realized in four jazz voicing styles (Four-Way Close, Drop 2, Drop 3, Drop 2+4). The pop dataset covers diatonic, modal, and borrowed-chord idioms across 83 unique progressions. Table 1 summarizes the combined dataset.

Each row is annotated along three orthogonal dimensions: **Emotion** (6 classes: Joyful, Vital, Epic, Uneasiness, Depressive, Despair), **Scale/Mode** (7 classes: Ionian, Aeolian, Dorian, Phrygian, Lydian, Mixolydian, Harmonic minor), and **Key** (12 major and 12 minor). Progressions are encoded in Roman numeral notation (*e.g.,* `I-V-vi-IV`), making the representation key-invariant: the model learns harmonic character, not absolute pitch. Pop progressions were expanded to all 12 keys via automated transposition using key-aware enharmonic spelling tables, ensuring balanced class coverage across all keys. Emotion and scale annotations follow established modal conventions. The six emotion classes were consolidated from thirteen original categories to cover a valence-arousal space.

Table 1. Dataset summary after preprocessing.

Source	Progressions	Rows	Coverage
Jazz	25	1,200	12 keys × 4 voicings
Pop	83	996	12 keys
Combined	**108**	**2,196**	–

3 Machine Learning Pipeline

3.1 Feature Design

The classification task is: *given (emotion, scale), predict the most probable chord progression type*, a 2-input, 108-class problem. Key is intentionally excluded from the feature vector because transposition does not alter harmonic character, and including it would fragment the training data into 24 identical shards. Both inputs are label-encoded to integer indices; the feature matrix is shape $[N \times 2]$ (float32), and the target indexes one of 108 progression classes. The dataset spans 23 of the 42 possible `(emotion, scale)` combinations; the model learns a distinct probability distribution over the 108 classes for each observed pair, enabling temperature-weighted sampling rather than a fixed argmax lookup.

3.2 Random Forest with Optuna Tuning

A Random Forest Classifier [3] is chosen because its `predict_proba()` output provides a per-class probability vector over all 108 progression types, directly usable as a stochastic sampling distribution at runtime; the downstream temperature scaling and weighted sampling (Section 4.3) act on this vector to enable probabilistic affective selection.

Hyperparameters are optimized with Optuna [4] using 50 trials of 5-fold stratified cross-validation. Training follows a two-stage procedure: hyperparameter search on an 80/20 stratified split, then retraining the final model on the complete 2,196-row dataset. The retraining step is critical: the split is stratified by *emotion* (6 classes), not by the 108-class progression target, so rare progressions may be absent from the 80% training fold, reducing the effective output dimension and breaking index mappings at runtime.

3.3 ONNX Export

The trained classifier is exported with `skl2onnx` [5] using `zipmap=False`, which produces a plain float32 tensor `"probabilities"` $[N, 108]$ enabling direct array access from C, and `target_opset=18` to match the bundled ONNX Runtime version. The resulting `gen_model.onnx` encodes the full Random Forest decision structure with no Python dependency. A companion vocabulary table `gen_data.tsv` (1,296 rows) maps progression IDs to concrete chord strings across 12 pitch classes, with major/minor spellings handled according to scale context.

4 Embedding ML in a Csound Opcode

4.1 Overview

The key contribution of AHM is that ML inference runs natively inside the Csound process. The authors built the plugin as a `.dylib` that links directly against the ONNX Runtime C library [6]; `emoChord_init SModelPath, SDataPath` loads the ONNX model and vocabulary table once at i-rate, storing all shared state in static C globals available to every subsequent `emoChord` call (not thread-safe across concurrent Csound instances). Figure 1 shows the two-phase architecture.

No external process is needed at runtime. Inference runs at i-rate; on Apple Silicon (arm64), median single-sample inference is 14 μs, far below typical score-event scheduling timescales, and model load is 52 ms.

4.2 Live Inference Pipeline in `emoChord`

`emoChord SEmotion, iInstr, iStart, iDur, iAmp [, iTemp, iOctave]` executes the following seven-step pipeline entirely inside the Csound process:

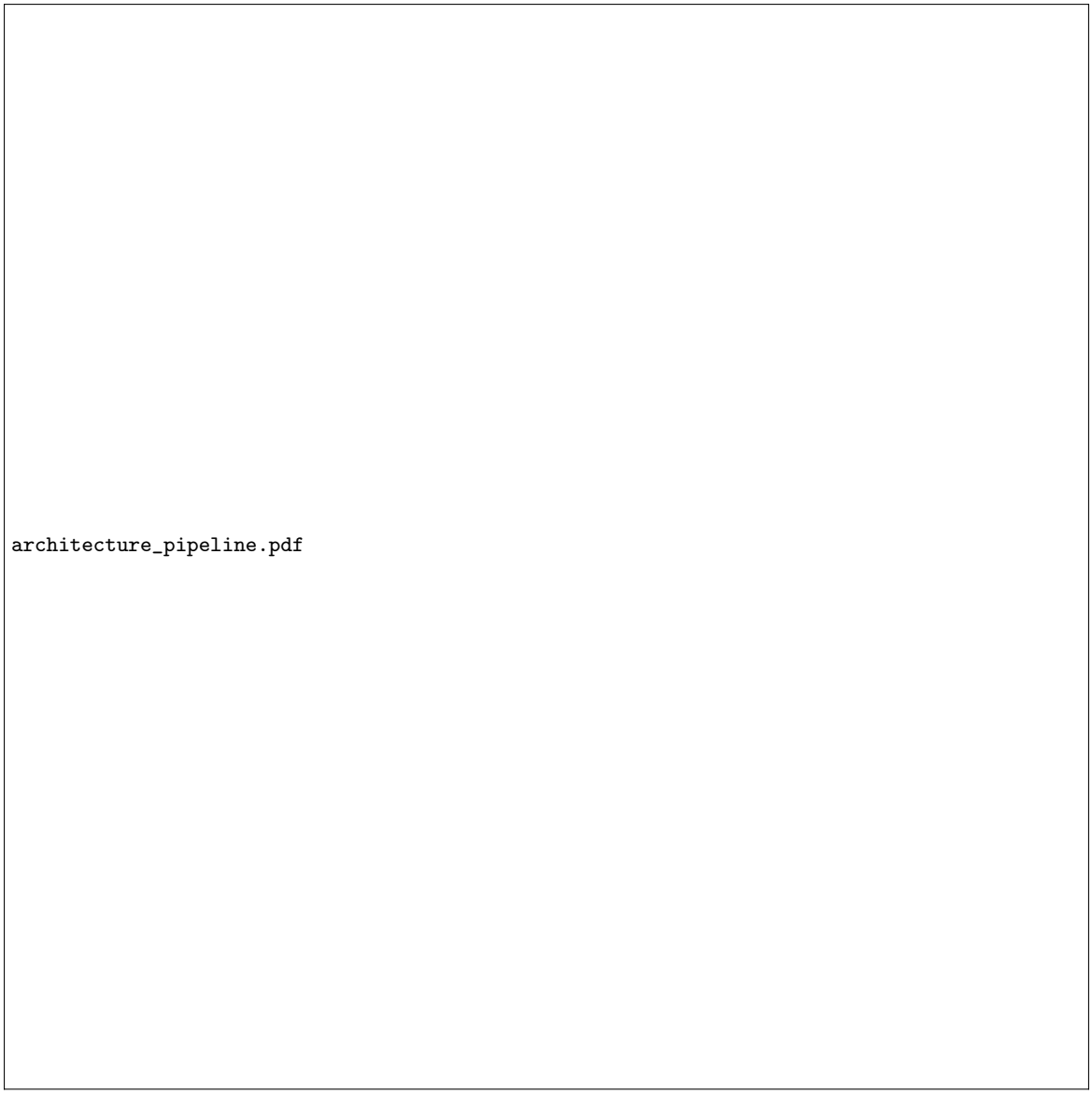

Fig. 1. AHM two-phase architecture. Offline (left): training and ONNX export of `gen_model.onnx` and `gen_data.tsv`. Runtime (right): native Csound inference and chord event scheduling via `emoChord`.

1. **Resolve emotion**: scan `DataEntry[]` for rows matching `SEmotion` (case-insensitive) to obtain `emotion_id`.
2. **Sample scale**: collect all `scale_id` values for that emotion, weighted by dataset frequency; draw one by temperature-weighted inverse-CDF.
3. **ONNX inference**: call `OrtApi->Run()` with input $[1, 2] = \{emotion\_id, scale\_id\}$, retrieving $\mathbf{p} \in \mathbb{R}^{108}$.
4. **Temperature scaling**: apply $p'_i = p_i^{1/T}$ and renormalize.
5. **Filter**: mask $\mathbf{p}'$ to `prog_id` values present for the sampled pair; guaranteed non-empty by construction, since Step 2 draws only from `scale_id` values observed for the given emotion.
6. **Sample progression**: draw one `prog_id` by weighted inverse-CDF.
7. **Schedule chords**: look up the `chord_name` string (*e.g.,* `Cm7-F7-Bbmaj7`), parse tokens into MIDI note numbers, and call `insert_score_event()` on instrument `iInstr`.

4.3 Temperature as Compositional Control

The `iTemp` parameter (default 1.0) translates the ML temperature concept into a direct musical control. At $T \to 0$ the opcode always returns the highest-probability progression; at $T = 1$ it follows the model's trained distribution; at $T > 1$ the distribution flattens toward uniform, increasing harmonic variety. A

composer can place two adjacent score events with the same emotion string but different temperatures: one to anchor a phrase and one to introduce a surprise chord, without touching the model or any other instrument parameter.

4.4 Usage Example

Listing 1 shows a minimal `.csd`. Instrument 1 reads an emotion string via `strget` and passes it to `emoChord`, which schedules events into any target instrument (here, Instrument 2).

Listing 1: Minimal `emoChord` usage.

```
<CsoundSynthesizer>
<CsOptions> -o dac </CsOptions>
<CsInstruments>
sr = 44100
ksmps = 32
nchnls = 2
0dbfs = 1
emoChord_init "/path/to/gen_model.onnx", "/path/to/gen_data.tsv"
instr 1  ; p4=emotion string
  Sem strget p4
  emoChord Sem, 2, p2, p3, 0.7
endin
</CsInstruments>
<CsScore>
i1  0  4  "joyful"
i1  6  4  "depressive"
i1 12  4  "uneasiness"
e
</CsScore>
</CsoundSynthesizer>
```

Passing the emotion as a score p-field follows the standard Csound idiom [8]. At each event, `emoChord` prints the selected progression (*e.g.,* `[emoChord] "joyful" -> Ionian -> C-G-Am-F`), making ML choices transparent during composition.

5 Results

The combined dataset totals 2,196 rows across 108 progression types, 6 emotions, and 7 scales; to our knowledge no public harmony corpus unifies jazz and pop under a six-class affect taxonomy.

After Optuna tuning (50 trials, 5-fold CV), the final model achieves a held-out top-1 accuracy of 24.8% and top-3 of 49.1%. A uniform random baseline yields 0.9% (1/108); a majority-class-per-pair baseline yields 24.8%, confirming that the Random Forest has faithfully learned the dominant harmonic preference per `(emotion, scale)` cell. The model is not presented as a high-accuracy predictor beyond the cell-level majority baseline; rather, its role is to package the learned class distribution into a runtime-compatible ONNX representation. The argmax case ($T \to 0$) is equivalent to that majority lookup; the model's added value over a fixed argmax lookup is that it preserves a full probability distribution that can be temperature-sampled at runtime.

An ablation at 477 progressions (4.4×) dropped top-1 to 10.6%: feature vectors within a cell are identical, bounding accuracy by inverse cell size, a ceiling set at dataset design time, not training time.

The 10 MB `gen_model.onnx` bundles with the plugin for fully offline deployment; all materials are at `https://github.com/CharlieSL1/AHM-Dataset`.

6 Conclusion

The central contribution of AHM is a reusable systems pattern: any ONNX-compatible classifier following the same input/output tensor contract can be embedded in Csound as a native i-rate opcode via the ONNX Runtime C API, with no Python or external-process dependency; substituting a different classifier requires no plugin source changes.

For the composer, affect becomes a score parameter alongside pitch and duration; the temperature control spans from the model's most confident prediction to open-ended harmonic exploration.

Future directions include a genre input to resolve jazz/pop cell ambiguity, longer chord sequence generation, a free-text prompt interface mapping natural-language input to the nearest `(emotion, scale)` pair, a k-rate variant, an `iKey` parameter, a perceptual evaluation, and Windows/Linux porting. Dataset and model improvements are ongoing; these results are an initial validated release. AHM establishes that small, domain-specific ML models are practical candidates for native Csound integration, offering a template for AI-augmented composition without cloud dependency.

References

1. Shi, L.C., Boulanger, R.: CStore: A Framework for Generating Editable, Versionable Csound Orchestra Code – From Text to Sound and Music. In: Proceedings of the 2026 IEEE Conference on Artificial Intelligence (CAI), pp. 237–243. IEEE, Granada (2026)
2. Lorrain, D.: A panoply of stochastic cannons. Computer Music Journal 4(1), 53–81 (1980)
3. Hevner, K.: The affective character of the major and minor modes in music. American Journal of Psychology 47, 103–118 (1935)
4. Pedregosa, F. et al.: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)
5. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631 (2019)
6. Dupré, X. et al.: sklearn-onnx: Convert scikit-learn models to ONNX. `https://onnx.ai/sklearn-onnx/` (2019–present)
7. ONNX Runtime Developers: ONNX Runtime inference engine. `https://onnxruntime.ai` (2018–present)
8. Lazzarini, V. et al.: Csound: A Sound and Music Computing System. Springer, Cham (2016)
9. Boulanger, R., Lazzarini, V. (eds.): The Audio Programming Book. MIT Press, Cambridge (2010)